

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as “the unacknowledged use, as one’s own work, of work of another person, whether or not such work has been published” (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

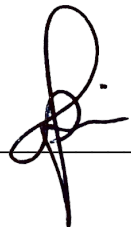
I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

David Farrugia
Student Name


Signature

Student Name

Signature

Student Name

Signature

Student Name

Signature

ICT5101
Course Code

ICT5101_IoT_Project_PartB_DavidFarrugia
Title of work submitted

06/06/2026
Date

Predictive Maintenance in Industrial IoT Using Machine Learning

David Farrugia

University of Malta

Msida, Malta

david.farrugia.22@um.edu.mt

Abstract

This project explores the use of machine learning for predictive maintenance in Industrial IoT environment. Using a synthetic dataset, the aim is to predict whether a machine would fail within the next seven days. Four models were compared: Logistic Regression, Random Forest, HistGradientBoosting, and XGBoost. The results showed that all models achieved strong ROC-AUC values, while HistGradientBoosting provided the best overall balance between accuracy, F1-Score, and generalisation. The project shows how ML can support failure detection and improve maintenance decision-making in Industrial IoT systems.

1 Introduction

Predictive maintenance is gaining popularity in the field of IoT (Internet of Things) and Industry 4.0 environments, especially in industrial environments where sensors can continuously monitor the condition of machines and equipment. Instead of waiting for a machine to fail, predictive maintenance uses data to identify signs of possible failure before they lead to downtime [12]. This makes it an important application of Industrial IoT, where sensor readings, operational data and machine information can support more informed maintenance decisions.

This project focuses on applying machine learning (ML) to a synthetic Industrial IoT dataset for predictive maintenance. Its main objective is to predict whether a machine is likely to fail within the next seven days. In this context, the problem is treated as a binary classification task, where each record is classified as either a failure case or a non-failure case.

The implemented solution follows a complete ML pipeline. The data is first loaded, cleaned and prepared for training. Some columns are removed to avoid data leakage, while additional features are engineered to better represent machine age, usage intensity, maintenance pressure and low oil or coolant levels before splitting into training, validation and testing sets. Stratification was also used to preserve the distribution of failure and non-failure cases.

Moreover, several models are trained and compared, including Logistic Regression, Random Forest, histogram-based gradient boosting (HistGradientBoosting) and XGBoost. Since failure cases are less common than non-failure cases, accuracy alone is not enough to evaluate the models' performance. For this reason, Precision, Recall, F1-Score, ROC-AUC and Average Precision are also calculated. Precision-Recall (PR) analysis is especially useful for imbalanced datasets because it focuses more directly on the model's ability to detect the positive class, making it better suited for this project since the positive class represents machine failure [5].

All code, datasets, experiments and related project files discussed in this report are publicly available on GitHub.¹

¹https://github.com/DavidF-22/ICT5101-IoT_Project/tree/main/PartB

2 Dataset and Problem Definition

The dataset used in this project is the synthetic Industrial IoT dataset provided on Kaggle[4]. It contains approximately 500,000 records of simulated machine sensor readings from a simulated smart factory environment. The dataset is designed for tasks such as predictive maintenance, failure forecasting and industrial machine monitoring, with each record representing the condition of a machine at a particular point in time, using operational, sensor-based and maintenance-related attributes.

Moreover, the dataset includes machine-related, operational, sensor-based and maintenance-related features, such as machine type, operational hours, temperature, vibration levels and maintenance history, among others. These variables provide information about both the current condition of the machine and its usage history. The target used in this project is `Failure_Within_7_Days`, which indicates whether the machine is expected to fail within the next seven days.

The problem is defined as a supervised binary classification task. Each record is assigned to one of two classes: failure within seven days or no failure within seven days. The aim is therefore to train machine learning models that can learn patterns from the available machine and sensor data and use them to classify unseen records. Since failure cases are less frequent than non-failure cases, the problem also involves class imbalance, which affects how the models should be evaluated.

3 Machine Learning Models

The selected models were chosen to compare a simple linear baseline with more complex tree-based ensemble methods. This is suitable for predictive maintenance since ML methods are commonly applied to maintenance prediction problems involving sensor and operational data [2]. Logistic Regression was included as a baseline model, while Random Forest, HistGradientBoosting and XGBoost were selected to evaluate whether ensemble-based models could better capture non-linear relationships between machine condition, usage patterns and failure risk.

3.1 Logistic Regression

Logistic Regression was used as a baseline binary classification model. It estimates the probability that a record belongs to the failure or non-failure class. Although it is simpler than the tree-based models, it is useful as a reference point because it allows the performance of more complex models to be compared against a simpler and more interpretable approach [9].

3.2 Random Forest

Random Forest is one of the ensemble methods chosen. It combines multiple decision trees instead of relying on a singular tree to produce a final prediction. This is achieved by aggregating the output

of many trees, which can help improve generalisation and reduce overfitting. Breiman describes Random Forests as a combination of tree predictors, where each tree depends on a randomly sampled vector [1]. In this project, Random Forest was selected because it can learn non-linear relationships between machine features and failure risk.

3.3 HistGradientBoosting

Moreover, HistGradientBoosting is a histogram-based gradient boosting classifier. While Gradient boosting builds models sequentially, where each new tree attempts to correct errors made by previous trees, histogram-based implementation improves efficiency by grouping continuous feature values into bins. This makes it suitable for larger tabular datasets. Additionally, scikit-learn notes that this estimator is much faster than standard gradient boosting for datasets with at least 10,000 samples [7]. Since the dataset contains approximately 500,000 records, this model was included as an efficient boosting-based classifier.

3.4 XGBoost

Finally, XGBoost. This machine learning method is an optimised gradient boosting framework based on decision trees, designed for scalability and performance on structured datasets. Chen and Guestrin describe XGBoost as a scalable tree boosting system that has been widely used for machine learning tasks [3]. In this project, XGBoost was included because the dataset is tabular and may contain non-linear relationships between operational conditions, sensor readings and future machine failure.

4 Methodology

The methodology in this project consists of an end-to-end machine learning pipeline for predictive maintenance. The pipeline includes data loading, preprocessing, feature engineering, dataset splitting, model training and model evaluation and interpretability.

First, the dataset was loaded using Python and inspected to understand its structure, data types and attributes before removing two columns from the dataset. The Machine_ID column was removed because it acts only as an identifier and does not provide meaningful general patterns for prediction. While the Remaining_Useful_Life_days column was also removed to avoid data leakage, since it is directly related to the target variable and could lead to unrealistically high performance. Furthermore, boolean columns, including AI_Supervision and Failure_Within_7_Days, were then converted to integer values so that they could be used by the machine learning (ML) models.

After cleaning, additional features were engineered to better represent machine condition and maintenance risk. Machine_Age was calculated from the installation year, while Operational_Hours_Per_Year was created to represent usage intensity relative to machine age. This was done, mainly since age is a meaningful indication of wear over time. Older machines may be more likely to experience component degradation, making this feature more relevant for failure prediction.

Furthermore, the Maintenance_Pressure feature was created by combining the number of days since the last maintenance with the previous failure history. This was done since a machine that has

not been maintained recently and has failed several times in the past may represent a higher risk than a machine with only one of these factors. Finally, two binary features, Low_Oil and Low_Coolant, were also added to indicate whether oil or coolant levels were below a selected threshold. These features were included because low oil or coolant levels can indicate poor conditions, which tends to contribute to increased wear and possible future failure.

The target variable was Failure_Within_7_Days, while the remaining columns were used as input features. The dataset was split into training, validation and testing sets using an 80/10/10 split respectively. The training set was used to fit the models, the validation set was used for model comparison during development and the test set was kept separate for final evaluation on unseen data. Since the target classes were imbalanced as seen in Figure 5 in Appendix B, stratified splitting was applied to keep the proportion of failure and non-failure cases across all three sets using the stratify parameter in scikit-learn's train_test_split function [10], which helps to minimise the effects of class imbalance.

Moreover, the categorical Machine_Type feature was converted into numerical form using one-hot encoding. This method was chosen since one-hot encoding proved ideal when testing with scikit-learn's one-hot and label encoders. After encoding, the validation and testing sets were aligned with the training set to ensure that all three sets contained the same feature columns. This made sure that the trained models received a consistent input structure during validation and testing.

During the implementation stage, several machine learning models were then trained and compared. Logistic Regression was used as a simpler baseline model, while Random Forest, Hist GradientBoosting and XGBoost were used as tree-based models capable of learning non-linear relationships. Each model was trained on the training set and evaluated on the validation set using Accuracy, Precision, Recall, F1-Score, ROC-AUC and Average Precision. The final model comparison was then performed on the unseen test set.

The model hyperparameters were selected as fixed baseline values using the official scikit-learn and XGBoost documentation as guidance. Exhaustive hyperparameter tuning was not performed, since methods such as Optuna could have significantly increased execution time. Therefore, the selected values were used to provide a practical and consistent configuration for comparing the models and to guarantee reproducibility.

For Logistic Regression, max_iter=1000 was used to allow more optimisation iterations, while class_weight="balanced" was used to reduce the effect of class imbalance. For Random Forest, n_estimators=150 trained a larger ensemble than the default setting, while min_samples_split=5 reduced overly specific splits. For HistGradientBoosting, max_iter=150, learning_rate=0.1 and max_leaf_nodes=31 controlled the number of boosting iterations, the contribution of each iteration and tree complexity. For XGBoost, n_estimators=150, learning_rate=0.1 and max_depth=6 were used as baseline boosting settings, with objective="binary:logistic" selected because the task is binary classification [7-9, 11].

Lastly, in the evaluation stage, along with metric calculations, permutation feature importance was also applied to the best performing model to improve interpretability. This method evaluates

the importance of each feature by randomly shuffling its values and measuring the resulting drop in model performance. Features that cause a larger performance drop are considered more influential in the model's predictions. This approach is useful for inspecting fitted models and understanding which features contribute most to predictive performance [6].

5 Evaluation and Results

The trained models were evaluated using Accuracy, Precision, Recall, F1-Score, ROC-AUC and Average Precision. Accuracy was included as a general measure of overall correctness. However, since the dataset is imbalanced, accuracy alone is not sufficient metric, since a model could achieve a high score by mainly predicting the majority class. Therefore, metrics such as Precision, Recall, F1-Score and Average Precision were given greater importance when comparing the models. Precision indicates how many predicted failures were actual failures, while Recall indicates how many real failures were detected. F1-Score provides a balance between Precision and Recall. Average Precision summarises the Precision-Recall curve, which is particularly relevant for imbalanced datasets where the positive class represents machine failure [5].

The final evaluation was performed on the unseen test set. As shown in Table 1 and Figure 1, all models achieved strong ROC-AUC values, with scores close to 0.98. This indicates that the models were generally able to separate failure and non-failure cases. However, Precision, Recall, F1-Score and Average Precision showed clearer differences between the models. Logistic Regression achieved the highest Recall and Average Precision, meaning that it detected most actual failure cases and ranked failure cases strongly. However, this came at the cost of lower Precision and a larger number of false positives, meaning that it was more aggressive in predicting failures.

Model	Acc.	Prec.	Rec.	F1	ROC-AUC	AP
HistGradientBoosting	0.965	0.724	0.673	0.697	0.983	0.742
Random Forest	0.963	0.689	0.706	0.697	0.981	0.713
XGBoost	0.964	0.714	0.672	0.692	0.983	0.745
Logistic Regression	0.929	0.458	0.971	0.623	0.983	0.752

Table 1: Test set performance summary

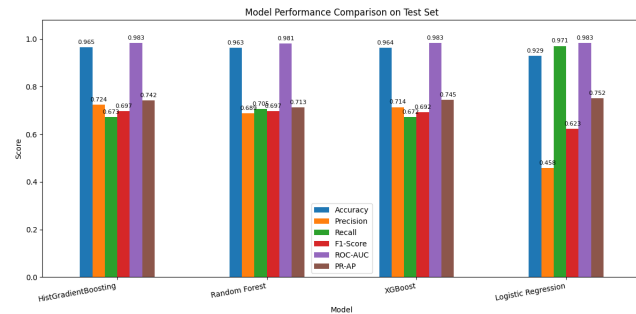


Figure 1: Model performance comparison on the unseen test set

In comparison, the tree-based models achieved a more balanced trade-off between Precision and Recall. HistGradientBoosting achieved the highest Accuracy and one of the strongest F1-Scores on the test set, while XGBoost achieved a slightly higher Average Precision

than HistGradientBoosting. Random Forest also performed competitively, with a similar F1-Score but a lower Average Precision. Overall, HistGradientBoosting was selected as the best-performing model because it provided a strong balance between overall accuracy, F1-Score and test-set generalisation.

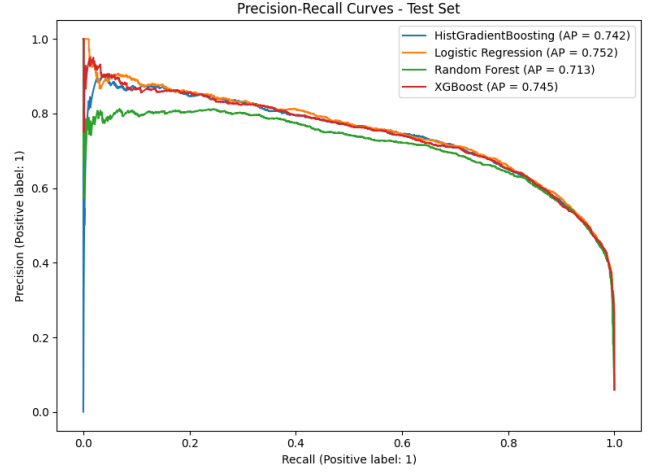


Figure 2: Precision-Recall curves for the trained models on the test set

The Precision-Recall curves in Figure 2 further show that the models performed similarly across many thresholds, although Logistic Regression and XGBoost achieved slightly higher Average Precision values on the test set. This supports the use of Precision-Recall in this project, since it focuses on the ability of correctly identify the failure class rather than being dominated by the majority non-failure class.

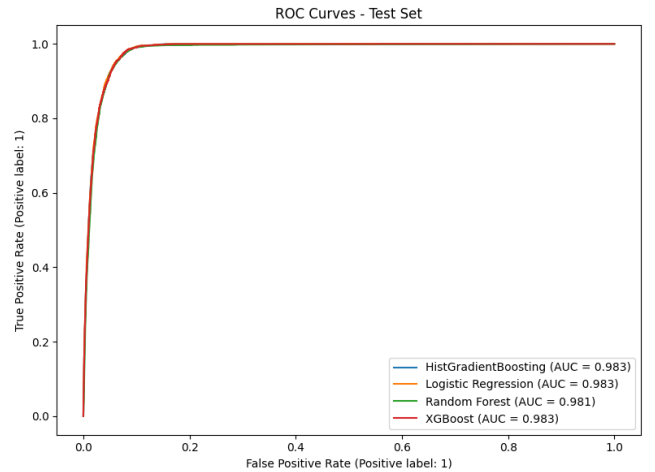


Figure 3: ROC curves for the trained models on the test set

Additionally, the ROC curves in Figure 3 show that all models were able to distinguish between failure and non-failure cases effectively, with ROC-AUC values between 0.981 and 0.983. The curves are closely grouped near the top-left corner, highlighting

that each model was able to distinguish failure and non-failure cases effectively across a range of thresholds. However, since the dataset is heavily imbalanced, ROC-AUC was not used on its own, and greater emphasis was placed on Precision-Recall.

When it comes to interpretability, Permutation feature importance was applied to the HistGradientBoosting model, since it was selected as the best overall model, to better understand which features contributed most to its predictions. As shown in Figure 4, Operational_Hours had the largest effect on Average Precision, followed by sensor and usage-related features such as Temperature_C, Vibration_mms and Operational_Hours_Per_Year. This suggests that the model relied mainly on operational usage and sensor condition indicators when predicting whether a machine was likely to fail within seven days.

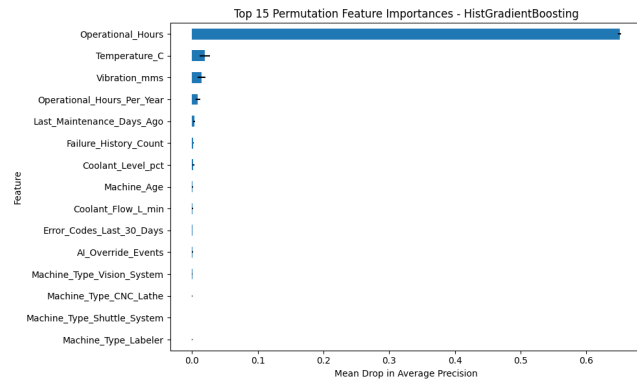


Figure 4: Permutation feature importance for the selected HistGradientBoosting model

Overall, the results show that all models separated failure and non-failure cases well, but Precision-Recall based metrics were more useful for comparing performance due to the class imbalance. Logistic Regression achieved the highest Recall but produced more false positives, while the tree-based models achieved a more balanced trade-off between Precision and Recall. HistGradientBoosting was selected as the best overall model because it achieved the highest Accuracy and a joint-highest rounded F1-Score while maintaining competitive Average Precision. Additional evaluation plots, such as confusion matrices, can be found in Appendix B.

6 Discussion

The results show that the models were generally able to distinguish between failure and non-failure cases, with all models achieving high ROC-AUC values. However, the Precision-Recall results gave a more useful comparison because the dataset is imbalanced and the failure class is the main class of interest. Logistic Regression achieved the highest Recall, meaning that it detected most failure cases, but this came at the cost of many false positives. In a predictive maintenance setting, this could lead to unnecessary maintenance checks.

The tree-based models provided a more balanced trade-off between Precision and Recall. HistGradientBoosting was selected as the best overall model because it achieved the highest Accuracy and a joint-highest rounded F1-Score, while still maintaining

good Average Precision. This suggests that it was able to generalise well on the unseen test set without being as aggressive as Logistic Regression in predicting failures.

Additionally, the feature importance results also showed that Operational_Hours had the strongest influence on the selected model. This is reasonable because machines with higher operating time are more likely to experience wear and degradation. Sensor-related features such as temperature and vibration were also among the more important features, which supports the idea that operational and condition-monitoring data are useful for predictive maintenance.

One limitation of this project however, is that the dataset is synthetic, meaning that the results may not fully represent real world industrial environments. In real-world settings, sensor noise, missing values, irregular maintenance records and unexpected operating conditions may affect model performance. Another limitation is that exhaustive hyperparameter tuning was not performed. Future work could therefore test the pipeline on real Industrial IoT data, apply more advanced tuning methods through Optuna for example, and explore threshold adjustment to better control the trade-off between false positives and missed failures.

7 Conclusion

This project implemented a machine learning pipeline for predictive maintenance using a synthetic Industrial IoT dataset[4]. The task was treated as a binary classification problem, where the aim was to predict whether a machine would fail within the next seven days. The pipeline included data cleaning, feature engineering, stratified dataset splitting, model training and model evaluation.

Four models were compared: Logistic Regression, Random Forest, HistGradientBoosting and XGBoost. The results showed that all models achieved strong ROC-AUC values, but Precision-Recall based metrics were more informative due to the imbalanced target classes. Logistic Regression achieved the highest Recall, while the tree-based models provided a better balance between Precision and Recall. Overall, HistGradientBoosting was selected as the best overall model because it achieved the highest Accuracy and a joint-highest rounded F1-Score on the test set.

In conclusion, the project shows that machine learning can be effectively applied to Industrial IoT sensor and operational data for predictive maintenance. Although the dataset was synthetic, the results demonstrate how predictive models can support early failure detection and help inform maintenance decisions.

Acknowledgements

This individual project was carried out as part of the partial fulfilment of the requirements of for the *ICT5101 Internet of things* course

References

- [1] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324
- [2] Thyago P. Carvalho, Fabrizzio A. A. M. N. Soares, Roberto Vita, Roberto da P. Francisco, João P. Basto, and Symone G. S. Alcalá. 2019. A systematic literature review of machine learning methods applied to predictive maintenance. *Computers & Industrial Engineering* 137 (2019), 106024. doi:10.1016/j.cie.2019.106024
- [3] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 785–794. doi:10.1145/2939672.2939785
- [4] Can Özensoy. n.d. Industrial IoT Dataset (Synthetic). <https://www.kaggle.com/datasets/canozensoy/industrial-iot-dataset-synthetic/data>. Kaggle dataset.
- [5] Takaya Saito and Marc Rehmsmeier. 2015. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE* 10, 3 (03 2015), 1–21. doi:10.1371/journal.pone.0118432
- [6] scikit-learn developers. 2026. Permutation feature importance. https://scikit-learn.org/stable/modules/permutation_importance.html. Accessed: 2026-06-02.
- [7] scikit-learn developers. 2026. sklearn.ensemble.HistGradientBoostingClassifier. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html>. Accessed: 2026-06-03.
- [8] scikit-learn developers. 2026. sklearn.ensemble.RandomForestClassifier. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>. Accessed: 2026-06-03.
- [9] scikit-learn developers. 2026. sklearn.linear_model.LogisticRegression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html. Accessed: 2026-06-03.
- [10] scikit-learn developers. 2026. sklearn.model_selection.train_test_split. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html. Accessed: 2026-06-02.
- [11] XGBoost developers. 2026. XGBoost Parameters. <https://xgboost.readthedocs.io/en/stable/parameter.html>. Accessed: 2026-06-03.
- [12] Tiago Zonta, Cristiano André da Costa, Rodrigo da Rosa Righi, Miromar José de Lima, Eduardo Silveira da Trindade, and Guann Pyng Li. 2020. Predictive maintenance in the Industry 4.0: A systematic literature review. *Computers & Industrial Engineering* 150 (2020), 106889. doi:10.1016/j.cie.2020.106889

A AI usage

Generative AI (GenAI) tools were used throughout this project, to support research guidance, debugging, report structure, wording refinement and checking whether the project remained in scope with this project and what was developed. The main tool used was ChatGPT and Gemini.

During the early stages of the project, GenAI tools were used to clarify concepts related to predictive maintenance, Industrial Internet of Things (IIoT) and evaluation metrics. This helped in understanding why metrics such as Precision, Recall, F1-Score, ROC-AUC and Average Precision were more suitable than relying only on Accuracy when evaluating the models.

GenAI tools were also used during implementation to support debugging and code review. In particular, these tools were useful when combining the model evaluation Precision-Recall and ROC-AUC plots. Some inconsistencies were present in the initial plots, and GenAI support helped identify possible causes and refine the plotting logic.

Lastly, in the report-writing stage, GenAI tools were used to improve the structure and flow of the report. They were also used to check whether the written content stayed within the scope of the project description and accurately reflected what was developed.

A.1 Prompts and Responses

Prompt 1: "Can you explain why Accuracy alone is not enough for an imbalanced predictive maintenance dataset?"

Summarised Response: The response explained that Accuracy can be misleading when the dataset contains many more normal

cases than failure cases. A model may achieve high Accuracy by mainly predicting the majority class, while still failing to detect actual machine failures. The response suggested giving more importance to Precision, Recall, F1-Score and Average Precision, since these metrics better reflect performance on the minority failure class.

Prompt 2: "Based on my project description and what I implemented, is this section still within scope? (The project description and source code were attached)"

Summarised Response: GenAI reviewed the draft section and identified which parts were aligned with the project. It also suggested areas where the wording could be improved to better match the implemented work, such as focusing on the machine learning pipeline, model comparison, evaluation metrics and the use of the unseen test set.

Prompt 3: "My PR-AUC and ROC-AUC plots have some issues when combining the model results into one figure. Can you help me debug the plotting logic? (The code block and errors were attached)"

Summarised Response: The AI tools helped review the plotting approach and suggested checking the model's outputs, to see if they were being stored and passed consistently. It also helped refine the combined plots so that the Precision-Recall and ROC-AUC curves were clearer and easier to compare across models.

A.2 Reflection

As a student who prefers hands-on implementation, I tend to find report writing and theoretical explanations very difficult as opposed to coding and experimentation. The use of GenAI tools helped make these parts of the project more manageable.

Additionally, GenAI tools were especially useful when refining the report structure and checking whether the content remained in scope with the assignment. They also helped during debugging, particularly when reviewing evaluation code and resolving inconsistencies in the generated plots.

However, it is worth mentioning that, all GenAI outputs were treated as support rather than final answers. Any suggested text, references, code changes or explanations were manually reviewed and adjusted to match the actual implementation and results of the project. This ensured that the final report reflected the work carried out and that the interpretation of the results remained my own.

B Additional Plots

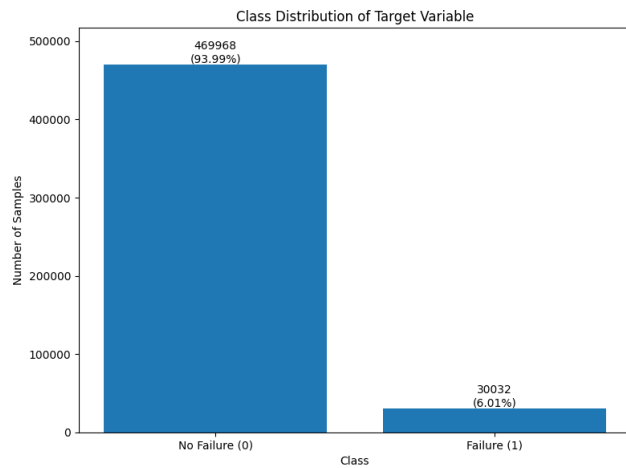


Figure 5: Dataset Distribution Failure and No Failure Cases

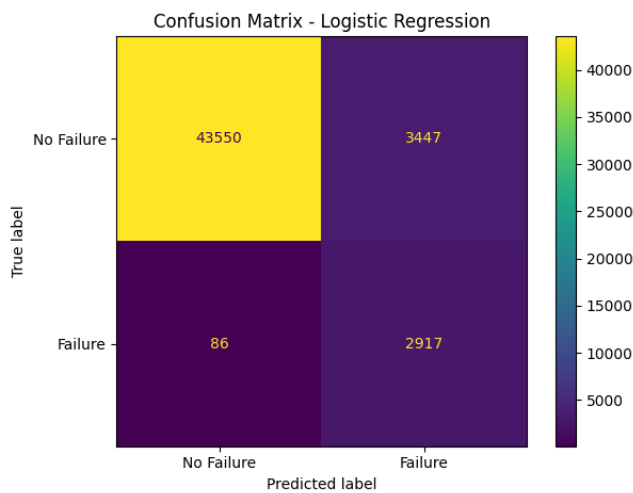


Figure 6: Confusion matrix for Logistic Regression on the test set

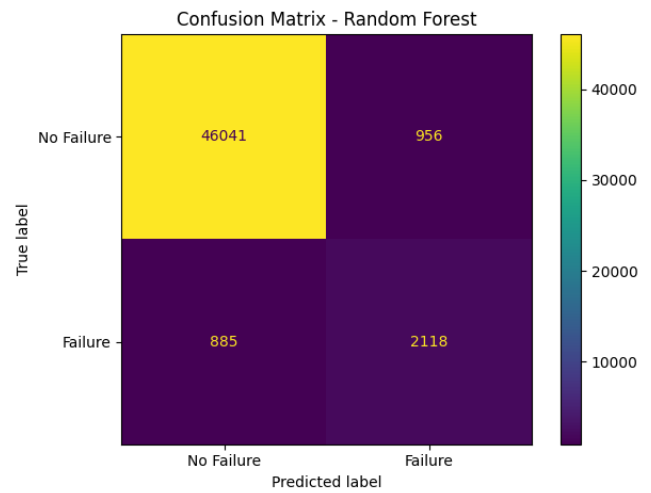


Figure 7: Confusion matrix for Random Forest on the test set

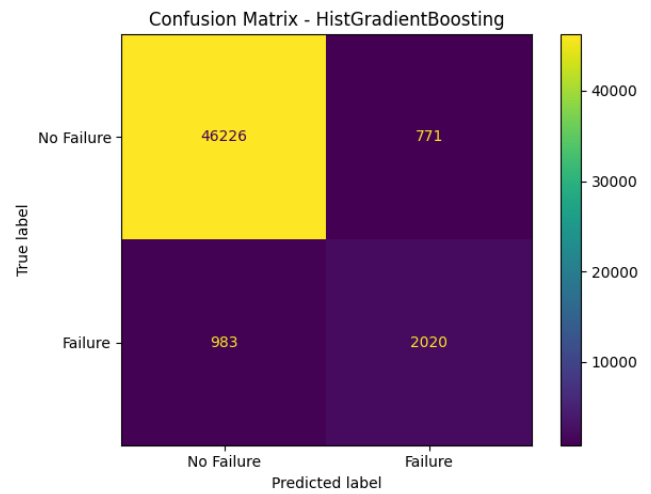


Figure 8: Confusion matrix for HistGradientBoosting on the test set

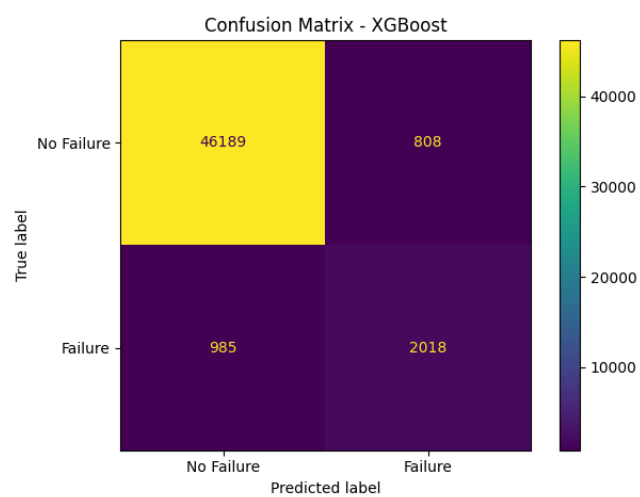


Figure 9: Confusion matrix for XGBoost on the test set